

# Delegates in C#



## Multicast Delegates

Part-2

Mr. Bangar Raju

 **NARESH**  
technologies

## Understanding Multicast Delegates in C#

By Albert Harmon

Published April 22, 2017

🕒 4 min read

Education



**TLDR** This blog post explains multicast delegates in C#, detailing their definition, usage, and practical implementation through examples. It covers the steps to define, instantiate, and invoke multicast delegates, emphasizing the importance of

# method signatures and the implications of using value-returning methods.

In the realm of C# programming, delegates serve as type-safe function pointers that allow methods to be called in a flexible manner. In this blog post, we will delve into multicast delegates, a powerful feature that enables a single delegate to reference multiple methods. This capability can simplify code and enhance functionality, especially when dealing with methods that share the same signature.

## What is a Delegate?

A delegate is essentially a type-safe function pointer that allows you to call methods. In C#, methods can be invoked in two primary ways:

1. **Using an instance of a class** for non-static methods.
2. **Using the class name** for static methods.

Delegates provide a third approach, allowing methods to be called through a delegate, which can reference both static and non-static methods.

## Steps to Use a Delegate

To effectively use a delegate, there are three essential steps:

1. **Defining a delegate**

## 2. Instantiating a delegate

## 3. Calling the delegate

In previous discussions, we focused on binding a method to a delegate and invoking it. Now, we will explore multicast delegates, which can hold references to multiple methods.

# What are Multicast Delegates?

Multicast delegates extend the functionality of standard delegates by allowing a single delegate to hold references to more than one method. This means that when a multicast delegate is invoked, all the methods it references are executed in the order they were added.

## Practical Example of Multicast Delegates

To illustrate the concept of multicast delegates, let's consider a practical example involving a `Rectangle` class that calculates both the area and perimeter of a rectangle.

### Step 1: Define the Rectangle Class

```
public class Rectangle
{
    public void GetArea(double width, double height)
    {
        Console.WriteLine(width * height);
    }
}
```

```
}

public void GetPerimeter(double width, double height)
{
    Console.WriteLine(2 * (width + height));
}
}
```

## Step 2: Define the Delegate

Next, we define a delegate that matches the signature of the methods we want to call:

```
public delegate void RectangleDelegate(double width, double height)
```



## Step 3: Instantiate the Delegate

Now, we create an instance of the `Rectangle` class and instantiate the delegate:

```
Rectangle rect = new Rectangle();
RectangleDelegate rectDelegate = new RectangleDelegate(rect.Ge
```



## Step 4: Binding Multiple Methods

Since both methods have the same signature, we can bind them to the same delegate:

```
rectDelegate += rect.GetPerimeter;
```

## Step 5: Invoking the Delegate

Finally, we can invoke the delegate, which will call both methods:

```
rectDelegate(12.34, 56.78);
```

This single call will output both the area and perimeter of the rectangle, demonstrating the power of multicast delegates.

## Important Considerations

When using multicast delegates, it is crucial to ensure that all methods referenced by the delegate have the same signature, meaning their return types and parameter types must match.

## Value-Returning Methods

If the methods are value-returning, only the result of the last method invoked will be captured, as it will overwrite the previous results. Therefore, it is advisable to use non-value-returning methods with multicast delegates to avoid losing results.

## Output Parameters

Similarly, if output parameters are used, the values will be overridden by the last method's output. To prevent this, it is best to avoid output parameters in methods that will be invoked through multicast delegates.

## Conclusion

Multicast delegates in C# provide a powerful way to invoke multiple methods with a single call, simplifying code and enhancing functionality. By understanding how to define, instantiate, and invoke these delegates, developers can leverage their capabilities effectively. Always remember to ensure that the methods share the same signature and avoid using value-returning methods or output parameters to prevent unintended behavior.

Thank you for reading! For more insights and tutorials, stay tuned to our channel.